

```
In [2]: # MODEL 1: Linear Regression

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

# Load data
url = "https://webpages.charlotte.edu/mschuck1/classes/DTSC2302/student-performance"
df = pd.read_csv(url)

# Define X and y
y = df["G3"]
X = df.drop(columns=["G3"])

# Identify column types
categorical_cols = X.select_dtypes(include=["object"]).columns
numeric_cols = X.select_dtypes(exclude=["object"]).columns

# Preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(drop="first"), categorical_cols),
        ("num", "passthrough", numeric_cols)
    ]
)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model1 = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

# Fit
model1.fit(X_train, y_train)

# Predict
y_pred1 = model1.predict(X_test)

# Evaluate
rmse1 = np.sqrt(mean_squared_error(y_test, y_pred1))
cv_rmse1 = -cross_val_score(model1, X, y,
                             scoring="neg_root_mean_squared_error",
                             cv=5).mean()
```

```
print("MODEL 1 RESULTS")  
print("Test RMSE:", rmse1)  
print("CV RMSE:", cv_rmse1)
```

MODEL 1 RESULTS

Test RMSE: 1.0846453501740916

CV RMSE: 1.3084204490550362

```
In [3]: feature_names = model1.named_steps["preprocessor"].get_feature_names_out()  
        coefficients = model1.named_steps["regressor"].coef_  
  
        coef_df = pd.DataFrame({  
            "Feature": feature_names,  
            "Coefficient": coefficients  
        })  
  
        print(coef_df.sort_values(by="Coefficient", ascending=False))
```

	Feature	Coefficient
41	num__G2	0.841030
5	cat__Mjob_health	0.315996
23	cat__higher_yes	0.274326
7	cat__Mjob_services	0.192554
17	cat__guardian_other	0.186102
40	num__G1	0.154760
8	cat__Mjob_teacher	0.139631
30	num__traveltime	0.105681
26	num__Unnamed: 0	0.101403
27	num__age	0.074838
2	cat__address_U	0.073808
24	cat__internet_yes	0.071157
19	cat__famsup_yes	0.066474
31	num__studytime	0.056910
29	num__Fedu	0.043667
39	num__absences	0.015750
21	cat__activities_yes	0.012602
36	num__Dalc	0.008778
33	num__famrel	0.004749
16	cat__guardian_mother	-0.005304
1	cat__sex_M	-0.012529
35	num__goout	-0.025595
3	cat__famsize_LE3	-0.036999
20	cat__paid_yes	-0.037186
38	num__health	-0.050962
13	cat__reason_home	-0.078456
34	num__freetime	-0.080568
37	num__Walc	-0.082526
22	cat__nursery_yes	-0.088250
28	num__Medu	-0.109450
4	cat__Pstatus_T	-0.125049
6	cat__Mjob_other	-0.139966
15	cat__reason_reputation	-0.144293
25	cat__romantic_yes	-0.152974
18	cat__schoolsup_yes	-0.159579
0	cat__school_MS	-0.243632
32	num__failures	-0.255413
10	cat__Fjob_other	-0.299521
9	cat__Fjob_health	-0.345803
11	cat__Fjob_services	-0.453458
14	cat__reason_other	-0.597633
12	cat__Fjob_teacher	-0.628265

In [4]: *# MODEL 2: Spline Regression*

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, SplineTransformer
from sklearn.pipeline import Pipeline

# Load data
```

```

url = "https://webpages.charlotte.edu/mschuck1/classes/DTSC2302/student-performance"
df = pd.read_csv(url)

# Define X and y
y = df["G3"]
X = df.drop(columns=["G3"])

# Identify column types
categorical_cols = X.select_dtypes(include=["object"]).columns
numeric_cols = X.select_dtypes(exclude=["object"]).columns

# Apply splines to selected variables
spline_features = ["absences", "studytime"]

spline_preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(drop="first"), categorical_cols),
        ("spline", SplineTransformer(degree=3, n_knots=4), spline_features),
        ("num", "passthrough", [col for col in numeric_cols if col not in spline_features])
    ]
)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model2 = Pipeline(steps=[
    ("preprocessor", spline_preprocessor),
    ("regressor", LinearRegression())
])

# Fit
model2.fit(X_train, y_train)

# Predict
y_pred2 = model2.predict(X_test)

# Evaluate
rmse2 = np.sqrt(mean_squared_error(y_test, y_pred2))
cv_rmse2 = -cross_val_score(model2, X, y,
                             scoring="neg_root_mean_squared_error",
                             cv=5).mean()

print("\nMODEL 2 RESULTS")
print("Test RMSE:", rmse2)
print("CV RMSE:", cv_rmse2)

```

MODEL 2 RESULTS

Test RMSE: 1.0757390886335791

CV RMSE: 1.2989129898738756